

Cloud5 Serverless Image Processing – Written Report

Project Overview

We built a serverless image-generation pipeline on AWS. Users submit text prompts, a Lambda calls the OpenAI Images API, and outputs are stored in S3. An HTTP interface (Lambda Function URL/API Gateway) and a Rust CLI provide client access. CloudWatch automatically captures Lambda logs/metrics, and deployment is automated via GitHub Actions building/pushing the Lambda container to ECR.

Architecture

• Prompt Ingestion (Event-Driven):

- Prompts uploaded as JSON to `s3://<bucket>/prompt/`.
- S3 notification → SNS → Lambda (handler.main_handler routes SNS events to api_handler).

• HTTP Interface:

- Same Lambda exposes http_handler via Function URL/API Gateway: accepts `{"prompt": "<text>"}` and returns `{"image_base64": "...", "metadata": {...}}`.

• AI Generation:

- Module ai_helper wraps OpenAI Images API with retries and response validation; supports b64 or URL response; parses api_key from Secrets Manager (JSON or raw).

• Persistence:

- Generated images written to `processed/ai/` in the same S3 bucket. (Optional metadata file can be added later.)

• Security & Secrets:

- OpenAI key stored in Secrets Manager (name `cloud5/openai/api-key`); Lambda loads via `OPENAI_API_SECRET_NAME`. Temporary override: `OPENAI_API_KEY`.
- IAM: Lambda needs `secretsmanager:GetSecretValue`, S3 read/write, CloudWatch Logs; SNS invoke permission.

• **Container & Deployment:**

- Lambda packaged as container (public.ecr.aws/lambda/python:3.13) with deps via uv.
- GitHub Actions workflow builds and pushes to ECR (ai-lambda). Lambda update is manual (owner updates image in console) because of cross-account ownership.

• **Client (CLI):**

- Rust CLI posts prompts to the HTTP endpoint; saves out.png. No OpenAI key exposed to users.

Decisions & Rationale

- **Serverless + SNS/S3:** Event-driven, minimal ops, easy fan-out.
- **Single Lambda with dual entry (event + HTTP):** Reuse logic; simpler maintenance.
- **Secrets Manager over plaintext env:** Security and rotation; parsing updated to tolerate JSON or raw.
- **Containerized Lambda:** Deterministic deps, quick rebuild via Actions.
- **Rust CLI hitting our backend:** Users never see the OpenAI key.

Issues Encountered

- **OpenAI org verification:** Blocked gpt-image-1 until org is verified.
- **Key formatting:** Secrets stored as JSON vs raw; added parsing to support both.
- **Cross-account deploy:** GitHub deploy user couldn't update Lambda; removed auto-update step—Lambda owner must set image manually.
- **IAM gaps:** Needed secretsmanager:GetSecretValue for Lambda role; added.

Key Learnings

- Secrets formatting matters—code should tolerate raw/JSON; verify with a local test before deployment.
- Cross-account CI needs either proper roles or a manual handoff step.

- Deploy pipelines should separate build/push from function updates when ownership differs.
- OpenAI org verification can block image models even with valid keys—check account status early.